

ChefVision AI

Project Workbook

By

Girith Choudhary

Richard Pham

Shivansh Chhabra

March 4, 2026

Advisor: Wencen Wu

Literature Search, State of the Art

Literature Search

ChefVision AI sits at the intersection of:

1. **Document understanding** (invoice/receipt extraction),
2. **Entity normalization** (ingredient name standardization / record linkage), and
3. **Forecasting + planning** (multi-horizon demand prediction + event-based ingredient planning).

Key research themes we are leveraging:

- **Visual Document Understanding (VDU) and Receipt Parsing**
 - Transformer-based document models reduce manual feature engineering and can learn layout + text jointly ([R1], [R2], [R3]).
 - Receipt datasets and competitions provide benchmarks for line-item extraction and key field extraction ([R4], [R5]).
- **Entity Matching / Normalization**
 - “Same entity, different name” is a core data management problem; deep learning + embeddings improve matching robustness over rules alone ([R6]).
 - ChefVision’s normalization layer combines similarity scoring, rules (units/vendor patterns), and (optional) human-in-the-loop confirmation.
- **Forecasting (Short-Term + Multi-Horizon)**
 - Practical forecasting systems emphasize robustness, seasonality, and evaluation workflows ([R7]).
 - Deep multi-horizon models (attention-based) can incorporate static covariates and known future inputs ([R8]).
 - Gradient boosting remains strong for tabular forecasting features and cold-start regimes ([R9], [R10]).

How the literature maps to our 3-model plan:

- **Model 1 (Transformer):** Document extraction from invoice/receipt images ([R1], [R2], [R3], [R4], [R5])
- **Model 2 (ML):** Recipe–ingredient modeling for usage/costing knowledge ([R11] plus curated recipes)
- **Model 3 (ML + planning logic):** Forecasting + event-based ingredient requirements ([R7], [R8], [R9], [R10])

State-of-the-Art Summary (Industry)

Modern “Document AI” has moved beyond basic OCR into **structured extraction** (tables, key-value pairs, line items) and **domain-specific parsers**. Leading-edge approaches increasingly combine:

- **Multimodal Transformers** (text + layout + vision) for robust extraction across formats
- **OCR-free or OCR-light pipelines** to reduce error propagation and simplify engineering
- **Human-in-the-loop review** for low-confidence fields to keep accuracy high at reasonable cost
- **Entity resolution + embeddings** to normalize noisy vendor naming and unify catalogs
- **Multi-horizon forecasting** to incorporate seasonality, local signals, and event schedules

In restaurant operations, existing inventory platforms primarily focus on inventory counts, ordering workflows, and recipe costing. ChefVision AI differentiates by making **photo-first ingestion + ingredient normalization + event-based planning** the center of the workflow:

- Start with photos/CSVs in minutes (low onboarding friction)
- Normalize supplier naming differences into a canonical ingredient catalog
- Convert events and recipes into **ingredient requirement plans** (event-by-event or week-by-week)
- Generate actionable reorder recommendations and shortage alerts

References

[R1] Geewook Kim, Teakgyu Hong, Moonbin Yim, JeongYeon Nam, Jinyoung Park, Jinyeong Yim, Wonseok Hwang, Sangdoo Yun, Dongyoon Han, Seunghyun Park. “**OCR-free Document Understanding Transformer (Donut)**.” *European Conference on Computer Vision (ECCV)*, 2022. (Also available as arXiv:2111.15664).

Relevance: Core “photo → structured invoice line items” model direction for your invoice/receipt ingestion.

[R2] Yupan Huang, Tengchao Lv, Lei Cui, Yutong Lu, Furu Wei. “**LayoutLMv3: Pre-training for Document AI with Unified Text and Image Masking**.” arXiv:2204.08387, 2022. (arXiv)

Relevance: Strong layout-aware Document AI baseline for extracting fields/line-items from invoices and receipts.

[R3] Sidharth Mudgal, Han Li, Theodoros Rekatsinas, AnHai Doan, Youngchoon Park, Ganesh Krishnan, Rohit Deep, Esteban Arcaute, Vijay Raghavendra. “**Deep Learning for Entity Matching: A Design Space Exploration**.” *SIGMOD 2018*, pp. 19–34. DOI: 10.1145/3183713.3196926. (DBLP)

Relevance: Direct foundation for your “same ingredient, different names” normalization/entity-resolution layer.

[R4] Bryan Lim, Sercan O. Arik, Nicolas Loeff, Tomas Pfister. “**Temporal Fusion Transformers for Interpretable Multi-horizon Time Series Forecasting.**” arXiv:1912.09363, 2019. (arXiv)

Relevance: Fits your phase-2 demand forecasting / multi-horizon planning (week-by-week reorder guidance + exogenous signals).

Project Justification

ChefVision AI solves a common, high-impact operational problem: **inventory decisions are only as good as the data behind them**, and that data is often unstructured (paper invoices, inconsistent item naming, scattered notes).

Why this matters:

- **Financial impact:** Mis-ordering drives waste, margin loss, and staff time spent reconciling invoices manually.
- **Operational reliability:** Stockouts create lost sales and degraded customer experience, especially during demand spikes.
- **Complexity increases over time:** More suppliers, changing pricing, and varying naming conventions create long-term data fragmentation.

Why ChefVision AI is the right approach:

- **Photo-first ingestion** matches the real workflow (owners already receive paper invoices and take photos).
- **Normalization is the multiplier:** Once “tomato / tomatoh / roma tomato” becomes one canonical ingredient, every downstream function improves (inventory accuracy, costing, forecasting).
- **Event-based planning is actionable:** Restaurants often plan around catering/events; converting events + recipes into ingredient requirements creates immediate value even before forecasting matures.
- **Scales from MVP to product:** Start with ingestion + normalization + Excel outputs; expand to integrations and predictive analytics as data accumulates.

Success criteria (high-level):

- Reduce manual invoice processing time
- Improve ordering accuracy (fewer stockouts and less overstock)
- Provide clear, explainable reorder suggestions tied to inventory + planned events

Project Requirements

Requirements (Use Cases / User Stories)

Essential (MVP)

- **Invoice Capture & Upload**
 - As a restaurant owner, I can take photos of invoices/receipts and upload them to the app.
 - As the system, I can store original images and extracted results with timestamps and vendor metadata.
- **Document Extraction (Model 1)**
 - As the system, I can extract vendor name, date, totals, and line items (ingredient name, qty, unit, unit price, total price).
 - As the restaurant owner, I can review extracted line items and correct errors.
- **Ingredient Normalization (Core AI)**
 - As the system, I can detect likely duplicates and spelling variants (e.g., tomato vs tomotoh) and map them to one canonical ingredient.
 - As the restaurant owner, I can confirm or override suggested matches when confidence is low.
 - As the admin, I can manage the canonical ingredient catalog and synonym rules.
- **Inventory Sheet Output**
 - As the restaurant owner, I can export/refresh an Excel-compatible inventory ledger (or spreadsheet view) with cleaned items and quantities.
- **Event + Recipe Planning**
 - As the restaurant owner, I can create upcoming events and enter: event name/date, recipes/menu items, and serving count (e.g., 100 people).
 - As the system, I can compute total ingredient requirements per event and aggregate week-by-week.
- **Smart Reorder List**
 - As the system, I can compare current inventory vs event requirements and generate a reorder list for shortages.
 - As the restaurant owner, I can see reorder quantities and the ingredient breakdown driving each recommendation.
- **Roles / Two Faces**
 - As the platform admin, I can access admin-only configuration screens.
 - As a restaurant owner/user, I can access only my restaurant workspace, invoices, and planning tools.

Desired (Next Iteration)

- **Forecasting (Model 3)**
 - Predict short-term demand using historical usage + calendar effects; recommend purchase quantities for a defined horizon (e.g., 7–14 days).
- **Supplier/Vendor Analytics**
 - Track price history, detect price spikes, and suggest cheaper vendor alternatives (where data supports it).

- **Menu/Recipe Costing**
 - Compute per-recipe ingredient cost and trend it over time.
- **Integrations (Planned)**
 - POS and supplier integrations via API/file drops.

Optional (Stretch)

- Automatic purchase order generation and email/send-to-vendor workflow
- Multi-location chain support and cross-store demand signals
- Waste tracking + anomaly detection for unusual usage patterns
- “What-if” planner (change guest count, swap recipes, see reorder deltas)

Non-Functional Requirements

- **Accuracy targets (initial):**
 - Line item extraction: target high precision on totals + quantities; allow user review for edge cases.
 - Normalization: high-confidence auto-merge; low-confidence requires confirmation.
- **Performance:**
 - Invoice processing completes within acceptable user wait time (or async with notifications).
- **Security & privacy:**
 - Role-based access control, encrypted storage, audit logs.
- **Reliability:**
 - Data integrity for inventory ledger; versioning for corrected extractions.
- **Usability:**
 - Photo upload should work with typical phone camera quality and imperfect lighting.

Dependencies and Deliverables

Dependencies (Showstoppers)

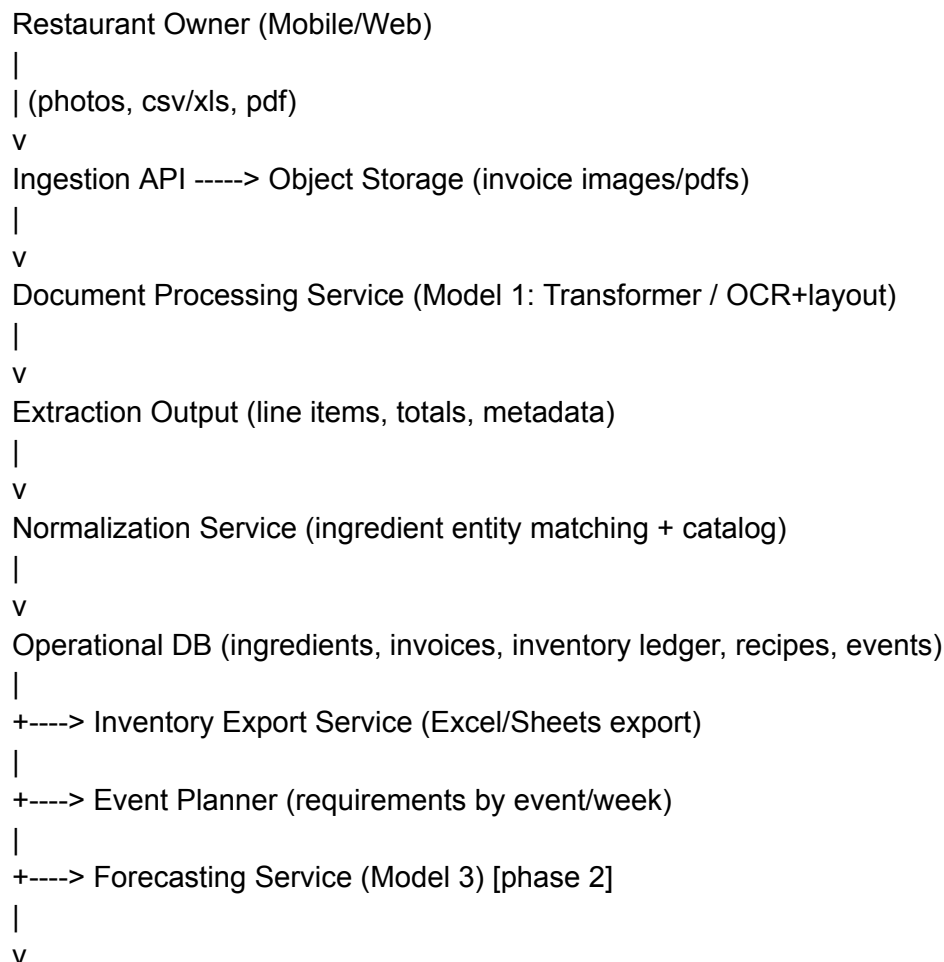
- Access to **representative invoices/receipts** for development/testing (real or sanitized).
- A **baseline ingredient catalog** (seed data) to support normalization early.
- Compute environment for model training/inference (GPU helpful for doc models).
- Storage for document images and extracted structured data.
- Recipe–ingredient dataset availability (public dataset + optional curated restaurant recipes).

Deliverables (Project-Specific)

- **Restaurant Owner App (User UI):** upload invoices, review extraction, inventory view/export, event planner, reorder list
- **Admin Console (Admin UI):** canonical ingredient catalog management, synonym rules, model/config controls
- **Backend API + Database:** authentication, data storage, business logic
- **ML Services**
 - Model 1: document extraction service
 - Normalization service: matching + dedup + confidence scoring
 - Model 3: forecasting (when enabled)
- **Excel/Spreadsheet Export Template** for inventory and planning outputs
- **Demo Dataset + End-to-End Demo** showing photo → inventory → event plan → reorder list

Project Architecture

Diagram (text version)



Reorder Recommendation Engine

|

v

Restaurant Owner UI (reorder list + explanations)

Admin (Web Console)

|

v

Catalog + Rules + Monitoring

One-page architecture description

ChefVision AI uses a client-server architecture. Users upload invoices as photos (primary ingestion) and optionally upload files (CSV/Excel/PDF). Documents are stored in object storage, then processed by a Document AI service (Model 1) that extracts structured line items. Extracted items are passed into a normalization service that resolves naming variants to a canonical ingredient catalog using embeddings + rules + confidence scoring. Cleaned inventory updates are stored in the operational database and can be exported to an Excel-compatible sheet.

On top of this cleaned data layer, the application supports event planning: users specify events, recipes, and serving counts; the system calculates ingredient requirements and checks inventory sufficiency. If shortages exist, the reorder engine generates a smart reorder list with explainable quantities (tied to event plans and historical usage). In later phases, a forecasting service (Model 3) provides short-term demand estimates to refine reorder quantities beyond events.

Project Design

Core Design Artifacts (text-first; diagrams can be converted later)

Use Cases (High Level)

- UC1: Upload invoice photo
- UC2: Extract invoice line items (auto)
- UC3: Review/correct extracted items (human-in-loop)
- UC4: Normalize ingredient names to canonical catalog
- UC5: Export/refresh inventory spreadsheet
- UC6: Create event plan (recipes + servings)
- UC7: Compute ingredient requirements and shortages
- UC8: Generate reorder list and weekly planning summary
- UC9: Admin manages catalog + synonym rules + confidence thresholds

Sequence (Invoice → Inventory)

1. User uploads invoice photo
2. Ingestion API stores image, creates processing job
3. Document Processing extracts fields + line items
4. Normalization matches ingredients to canonical catalog
5. System writes cleaned items to DB and updates inventory ledger
6. UI shows results + lets user confirm corrections
7. Export service updates Excel output

Sequence (Event → Reorder List)

1. User creates events and selects recipes + serving counts
2. System expands recipes into ingredient quantities (Model 2 knowledge + recipe table)
3. System aggregates requirements event-by-event and optionally week-by-week
4. System compares requirements vs inventory ledger
5. System generates reorder list + shortage explanations

Data Model (Entities)

- User(id, role, restaurant_id, ...)
- Restaurant(id, name, settings, ...)
- Supplier(id, name, contact, ...)
- Invoice(id, restaurant_id, supplier_id, date, totals, image_uri, status, ...)
- InvoiceLineItem(id, invoice_id, raw_name, raw_qty, raw_unit, raw_price, ...)
- IngredientCanonical(id, canonical_name, category, unit_defaults, ...)
- IngredientAlias(id, canonical_id, alias_text, vendor_context, ...)
- NormalizationMatch(id, line_item_id, canonical_id, confidence, decided_by, ...)
- InventoryLedger(id, restaurant_id, canonical_id, on_hand_qty, last_updated, ...)
- Recipe(id, name, yield, unit, ...)
- RecipeIngredient(id, recipe_id, canonical_id, qty_per_serving or qty_per_batch, ...)
- Event(id, restaurant_id, name, date, servings, notes, ...)
- EventRecipe(id, event_id, recipe_id, servings, ...)
- ReorderRecommendation(id, restaurant_id, date, canonical_id, recommended_qty, reason, ...)

UI Mockups (Screen List)

Restaurant Owner (User):

- Login / Restaurant selection
- Upload invoice (camera/gallery)
- Extraction review (line items editable)
- Ingredient matches (confirm/override)
- Inventory dashboard (search + export)
- Events planner (events list + event detail)

- Requirements summary (event-by-event, week-by-week)
- Reorder list (quantities + explanation)

Admin:

- Canonical ingredient catalog manager
- Alias/synonym rules + match thresholds
- Model monitoring dashboard (accuracy metrics, failure cases)
- Restaurant onboarding/settings

QA, Performance, Deployment Plan

Testing Plan

- **Unit tests**
 - Normalization logic (string + embedding match, confidence thresholds)
 - Inventory math (units, conversions, aggregation)
 - Event requirement calculations and reorder list generation
- **Integration tests**
 - Photo upload → extraction job → normalized output in DB
 - Export generation correctness (Excel output matches DB state)
- **Model evaluation**
 - Document extraction: field-level precision/recall/F1 on a labeled invoice set
 - Normalization: match accuracy + human override rate
 - Forecasting (phase 2): MAPE/SMAPE over rolling backtests
- **User acceptance tests**
 - End-to-end workflow: invoice → cleaned inventory → event → reorder list

Performance Plan

- Measure processing time per invoice and identify bottlenecks (upload, extraction, normalization).
- Load testing for peak upload times (batch invoice processing).
- Ensure async processing works reliably with job status + retries.

Deployment Plan

- Containerized services (API, workers, ML inference) deployed to a cloud environment.
- CI/CD for automated test runs and staged deployments.
- Monitoring:

- Logs, latency metrics, error rates
 - Model drift checks and extraction failure tracking
- Data backups and retention policies for invoices and ledger history.

Implementation Plan and Progress

Implementation Plan (Phased)

Phase 0: Setup

- Repo + branching strategy, environment setup
- DB schema draft, API skeleton, basic auth/roles

Phase 1: Ingestion + Extraction MVP

- Mobile/web upload flow
- Baseline extraction pipeline (Model 1 integration)
- Storage + job queue + retry logic
- UI for reviewing extracted line items

Phase 2: Normalization + Inventory Ledger

- Canonical ingredient catalog + alias table
- Matching logic (embeddings + rules)
- Inventory ledger updates + Excel export

Phase 3: Recipe + Event Planning

- Recipe/ingredient tables + serving scaling
- Event planner UI
- Requirements computation and reorder list output

Phase 4: Forecasting

- Feature engineering from ledger + calendar + events
- Baseline models (boosting/Prophet) then upgrade as data grows
- Backtesting + monitoring

Current Progress (as of March 4, 2026)

- Project scope defined: photo-first invoice ingestion → normalization → inventory sheet → event planning → reorder list.

- Three-model roadmap established (Document AI, recipe/ingredient model, forecasting + planning engine).
- Kickoff checkpoint recorded: 2/20/2026, 1:15 PM (“Letsgo”).

Project Schedule

Sprint Plan (Example: 10-week plan starting Mar 4, 2026)

Sprint 0 (Mar 4 – Mar 10): Foundations

- Define MVP scope, user stories, acceptance criteria
- Set up repo, CI, dev environments, initial DB schema

Sprint 1 (Mar 11 – Mar 24): Ingestion + Job Pipeline

- Photo upload + storage
- Processing job queue + status tracking
- Stub UI for invoice list + upload

Sprint 2 (Mar 25 – Apr 7): Document Extraction (Model 1)

- Integrate extraction model/service
- Store extracted line items in DB
- Review UI (edit line items)

Sprint 3 (Apr 8 – Apr 21): Normalization + Catalog

- Canonical ingredient catalog + alias management
- Matching + confidence scoring
- “Confirm match” UI and audit trail

Sprint 4 (Apr 22 – May 5): Inventory Ledger + Excel Export

- Ledger updates from normalized line items
- Excel export generator + downloadable report
- Basic analytics (top spend, frequent ingredients)

Sprint 5 (May 6 – May 19): Event Planner + Reorder List

- Events + recipes + serving count
- Ingredient requirement calculator
- Reorder list generation (event-by-event + weekly summary)

Sprint 6 (May 20 – Jun 2): Hardening + Demo

- QA pass, performance tuning, bug fixes
- Final demo script and documentation

Team Task Allocation (replace with real names)

- **Richard Pham:** Document extraction (Model 1), dataset prep, evaluation metrics
- **Girith Choudhary:** Ingredient normalization service, canonical catalog design, matching evaluation
- **Shivansh Chhabra:** Full-stack integration (UI/API), deployment pipeline, Excel export, testing harness